Response to the committee

A Lightweight Canonicalization Engine in C for Disciplined Convex Programming

William Zijie Zhang (2362162), Master's thesis, Electrical Engineering, Polytechnique Montréal

Supervisor: Antoine Lesage-Landry

October 26, 2026

To Sébastien Le Digabel (chair), Antoine Legrain (second member) and Antoine Lesage-Landry (supervisor).

Message au jury

Bonjour chers membres du jury,

Je voulais tout d'abord vous remercier pour vos commentaires sur mon mémoire. Comme vous m'avez demandé de clarifier et d'approfondir plusieurs points des résultats, j'ai pris le temps de les rendre plus complets, et j'ai aussi découvert quelques erreurs dans les affirmations que j'avais faites sur les contributions.

Premièrement, il y a maintenant quatre problèmes qui sont plus lents. J'ai ajouté une section du mémoire pour les décrire (§4.4). Trois des quatre problèmes peuvent devenir plus rapides. Deux d'entre eux le deviennent avec des changements dans le « frontend » (par frontend, je fais référence à la manipulation des arbres d'expression avant le calcul des dérivées). Par exemple, pouvoir détecter que `D @ ones` (où `D` est une matrice quelconque) équivaut à `D.sum(axis=1)`, la somme sur les colonnes, est un grand avantage. Le troisième, ConvexPlasticity, le devient avec un test de densité. Ces changements ne règlent pas tout : une partie du ralentissement vient du moteur lui-même, en particulier de la mémoire de la disposition globale (Q5).

Deuxièmement, j'ai refait toutes les mesures sur un ordinateur plus récent (MacBook M2 de 2022, avec Apple `Accelerate`), plutôt que sur l'ordinateur Intel de 2018 de la version soumise, pour que les résultats soient reproductibles sur du matériel actuel, comme celui de la plupart des utilisateurs de `CVXPY`. Pour la première canonicalisation, les ratios et les temps sont semblables. Pour les re-résolutions paramétriques, en revanche, `diffengine` n'est plus rapide que sur trois des sept problèmes, contre cinq dans la version soumise. Les nouvelles mesures par phase expliquent où va le temps : sur les quatre autres, la passe numérique ne prend que 4 à 24 % d'une re-résolution, et le reste sert à convertir les matrices au format CSC et dans l'ordre des cônes du solveur, alors que le tenseur de `CVXPY` produit directement ce format. J'ai donc modifié les affirmations de ma conclusion et de la discussion des résultats pour dire seulement qu'il y a des bénéfices, mais que la séparation ne remplace pas encore le tenseur pour les re-résolutions. Pour en trouver la cause, j'ai refait ces re-résolutions sur le M2 avec les versions des logiciels de la version soumise. Avec elles, `diffengine` re-résout encore plus vite que le tenseur sur cinq des sept problèmes, comme dans la version soumise : le changement ne vient donc pas de l'ordinateur, qui réduit l'écart sans l'inverser. Il ne vient

pas non plus du moteur, puisque l'ancienne version de `CVXPY` avec le moteur actuel donne les mêmes temps. Il vient surtout de l'interface entre `CVXPY` et le moteur, qui réordonne à chaque re-résolution les lignes de $\mathbf{A}$ dans l'ordre des cônes du solveur. L'ancienne version le faisait par un seul produit de matrices creuses ; une modification de `CVXPY` postérieure à la version 1.9.2 sélectionne plutôt les lignes, ce qui ne coûte rien au tenseur, réordonné une seule fois, mais oblige `diffengine` à convertir $\mathbf{A}$ du format CSC au format CSR et inversement à chaque re-résolution, soit environ 70 % d'une re-résolution sur SimpleScalarParametrizedLP. Les deux donnent les mêmes matrices. Comme les autres backends sont mesurés avec la version 1.9.2, qui n'inclut pas cette modification, les re-résolutions de `diffengine` dans le mémoire sont maintenant mesurées sans elle aussi ; la première canonicalisation, où le réordonnancement n'a lieu qu'une fois, ne change pas. Le reste de l'écart avec la version soumise vient d'autres changements de l'interface, et réduire ce coût de conversion est maintenant le premier des travaux futurs.

Troisièmement, le problème UnconstrainedQP n'était pas réellement 100 fois plus rapide. C'était dû à un changement de frontend que j'avais appliqué sur la branche du backend `diffengine`, mais pas sur celles des autres. En l'appliquant à tous, cet avantage disparaît. Ceci a été rectifié : aucun backend n'utilise maintenant ce changement.

À part ces trois points, qui sont les changements majeurs, j'ai aussi fait les études d'ablation et exécuté plus d'instances de benchmark, avec des mesures de signification statistique, ce qui répond à tous vos commentaires.

J'ai rédigé moi-même les réponses à vos questions. Claude, l'outil d'IA déclaré dans le mémoire, les a ensuite reformulées et mises en forme en anglais ; les chiffres viennent directement des scripts de mesure.

1 Summary of corrections

I have made all the modifications required by the committee and by Antoine Legrain's examiner report, as well as the minor corrections that Sébastien Le Digabel annotated and those listed in the appendix of Antoine Legrain's report. The minor corrections are not listed here. All changes since the version sent to the committee are marked in `Document-diff.pdf`, with added text in blue and deleted text in red. Section, table and page numbers below refer to the final version.

Table 1 lists the required modifications. "J" refers to the committee's requirements after the defence, and (i)–(vii) to the items of the synthesis of the examiner report. Each item is answered in detail, with the new results, in Sections 2 and 4.

Table 1: Required modifications and where they are made.

Request	Change	Where
J1a: statistical significance	Every timed comparison is made in 30 runs and tested per problem with a two-sided Mann–Whitney U test [1], Holm-adjusted [2] over the problems, and over the suite with a Wilcoxon signed-rank test [3] and a bootstrap confidence interval. Significant ratios are starred in the tables and filled in the figures.	Ch. 4 introduction, p. 51; §4.1
J1b, (ii): ablation	The dense blocks are turned off on all 25 problems, for time and memory, and on the re-solves together with the split (Q2).	§4.2, p. 57
J1c: more instances	Each run draws new parameter values from its own seed, and every problem is also scaled in size and density (J1d).	Ch. 4 introduction; §4.3
J1d: sensitivity to size and sparsity	All 25 problems at five sizes, $s \in \{1/4, 1/2, 1, 2, 4\}$, and eight problems at seven densities, with and without the dense blocks (Section 4.2).	§4.3, p. 60
J1e, (iii): non-DPP problems	Re-solves of two families in DPP, non-DPP and parameter-free variants, and of ConvexPlasticity, the suite's one non-DPP parametric problem (Q3).	§4.5.1, p. 66
J2, (i): related literature and DNLP	Matrix-free modelling, `CVXcanon`, `CVXGEN`, `Convex.jl` 0.16, `Pyomo`'s persistent interfaces, block-sparse antecedents (finite elements, `Ceres`, `g2o`), and sparse factorization as the origin of the symbolic/numeric split. The thesis now states who leads `SparseDiffEngine` and what is specific to this work (Q6).	§2.2, §1.5, §3.1.1
J3: declaration of AI use	New front-matter page.	Front matter
(iv): repeated measurements	30 runs, each in a fresh process; tables give mean ± standard deviation.	Ch. 4 introduction; Table 4.1
(v): dimensions of the instances	m, n, nnz($\mathbf{A}$), nnz($\mathbf{P}$), parameter size and DPP status of every problem.	Appendix A, Tables A.1 and A.2
(vi): time by phase	The share of each phase on the four slower problems, their formulations and the mechanism for each (Section 4.3).	§4.4, p. 62
(vii): correctness check	The problem data are checked against the `CPP` backend to a relative tolerance of 10^{-9}, at the first canonicalization and after a parameter update.	Ch. 4 introduction

A change of baseline. The version sent to the committee reported a 2.8× geometric-mean speedup of `diffengine` over the best `CVXPY` backend. Two things have changed since. First, all experiments were rerun on an Apple M2 with the current versions of `CVXPY` and `SparseDiffEngine`. Second, the earlier comparison was not fair: `diffengine` was timed with `CVXPY` pull request #3524, which reassociates chains of matrix products that end in a constant, while the other backends were timed without it. That pull request speeds up every backend, so now none of them uses it. In the same way, the re-solves of `diffengine` are timed without pull request #3525, which came after version 1.9.2: it reorders the rows of $\mathbf{A}$ into the cone order by a row selection instead of a product with a permutation matrix, which costs the tensor nothing, because it is reordered once, but costs `diffengine` a conversion of $\mathbf{A}$ to CSR and back at every re-solve. With this baseline, `diffengine` is the fastest of the four backends on 21 of the 25 problems, significantly on all 21, with a geometric-mean ratio of 0.44 to the best `CVXPY` backend, about 2.3× faster. UnconstrainedQP is now 11.7× slower; Section 4.3 explains why.

Methodology. Every timed comparison is made in 30 runs, each in a fresh process, on an Apple M2 (24 GB, macOS 26.5.1) with Apple's `Accelerate`, single-threaded. On each problem, the runs of `diffengine` and those of the other backend are compared with a two-sided Mann–Whitney U test [1], Holm-adjusted [2] over the problems of the comparison. Over the suite, we test the geometric mean of the ratios with a Wilcoxon signed-rank test [3] on the log-ratios and give its bootstrap 95 % confidence interval.

The usual rule of at least 30 observations comes from the normal approximation of a sample mean. The Mann–Whitney test is a rank test and assumes no normality, so it is valid at 20 runs as well; with 20 runs per side, its smallest Holm-adjusted p-value over 25 problems is already 1.7×10^{-6}. We nevertheless reran every comparison at 30 runs, so that the tables also give stable means and standard deviations.

2 Replies to the defence questions

Q1. *Why materialize dense blocks rather than leave them as operators, as in the matrix-free approach of Diamond and Boyd [4]? When is each approach preferable?*

Because the solvers that `CVXPY` calls need the matrices. Its default solvers are `Clarabel`, an interior-point method, and `OSQP` and `SCS`, which are first-order methods; commercial solvers such as `MOSEK` and `Gurobi` are interior-point or simplex methods. All of them factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$: an interior-point method factorizes a KKT system at every iteration, and `OSQP` and `SCS`, in their default direct mode, factorize their KKT matrix once and reuse the factorization across iterations. So every one of them needs $\mathbf{A}$ and $\mathbf{P}$ explicitly, whatever its algorithm. Matrix-free modelling pays off for very large problems built from fast transforms, solved by first-order methods that only need products with $\mathbf{A}$ and $\mathbf{A}^\top$, such as the indirect mode of `SCS`. A permuted dense block lies between the two: it is a materialized matrix, but it keeps the structure of the operator, namely its row and column selectors and

its dense block.

Changes in the thesis, §2.2, p. 24

In convex optimization, Diamond and Boyd take the matrix-free view. Instead of forming $\mathbf{A}$, they represent the problem as a graph of linear operators, available only through their forward and adjoint products, which suits first-order solvers. The solvers that `CVXPY` calls by default instead factorize a matrix built from $\mathbf{A}$ and $\mathbf{P}$. The matrix-free approach pays off on very large problems built from fast transforms, such as convolutions and Fourier transforms, and it needs solvers that accept operators instead of matrices. The permuted dense matrices of §3.2 sit between the two: they form the matrix that the solver factorizes, but keep the structure of the operator, its row and column selectors and its dense block.

Q2. *If the permuted dense representations are disabled, what fraction of the speedup remains?*

Most of it. Without the dense blocks, `diffengine` is 2.15× faster than the best `CVXPY` backend in geometric mean, against 2.29× with them. The difference between the two, 1.06×, is not significant over the suite (Wilcoxon signed-rank, $p = 0.07$). Per problem, the effect is significant on 24 of the 25 problems and large in both directions, as Figures 1 and 2 show. On six problems with dense data, the dense blocks take the engine from about 3 to about 7× faster than `CVXPY`. On UnconstrainedQP they take it from 4× faster to 11.7× slower, and on ConvexPlasticity from 1.4 to 7.3× slower, because both build a dense block from data that is sparse in fact.

The split between the symbolic and the numeric pass plays no part in a first canonicalization, because each pass runs once. It is ablated on the re-solves instead (Table 2). The dense blocks are the only part of the engine that can be turned off on a first canonicalization while keeping everything else the same. The other differences from `CVXPY`'s backends, namely the implementation in C, one Jacobian per node and the global layout, are built into every atom and cannot be switched off one at a time. We therefore do not attribute the remaining 2.15× to any one of them.

Figure 2 shows why the mean over the suite hides the effect: the dense blocks help by 1.8 to 3× on eight problems with dense data and hurt by 1.4 to 46× on the three problems with a dense block built from sparse data (CVaR, ConvexPlasticity, UnconstrainedQP). They also cost memory: without them, `diffengine` uses 0.83 times the peak memory in geometric mean, and CVaR falls from 10.7 to 2.1 GB. On the re-solves, the split is what matters, and the dense blocks do not (Table 2): without the split, a re-solve is 2.9× slower, and significantly so on all seven problems. *We conclude that most of the first-canonicalization speedup comes from the engine as a whole rather than from the dense blocks, which should only be built from data that is dense in fact, and that the re-solve speed comes from the split.*

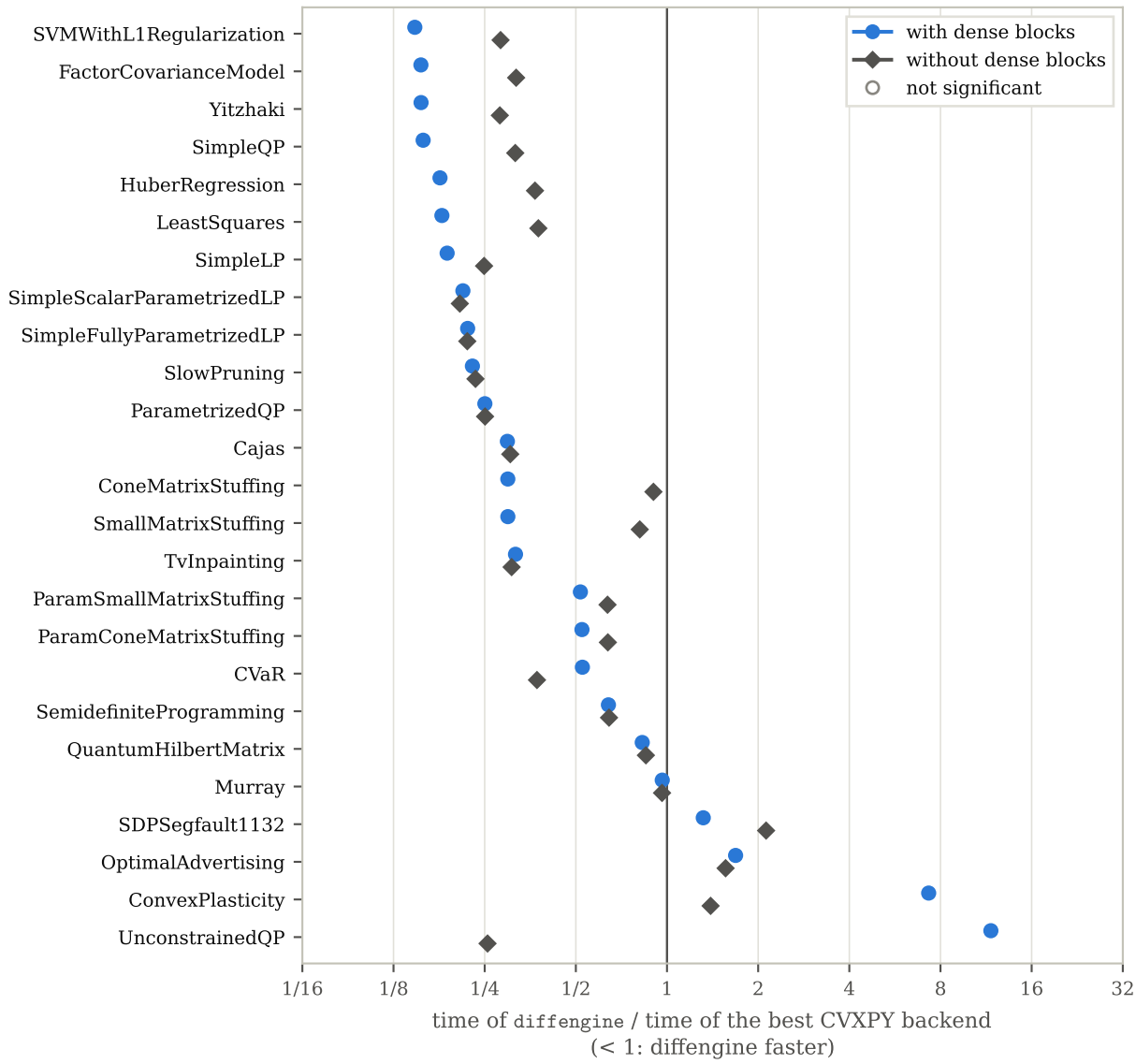


Figure 1: Figure 4.1 of the thesis (p. 55): ratio of `diffengine` to the best `CVXPY` backend on each problem, with its bootstrap 95 % confidence interval, with and without the dense blocks. Filled markers are significant (Mann–Whitney, Holm-adjusted $p < 0.05$); here every one is.

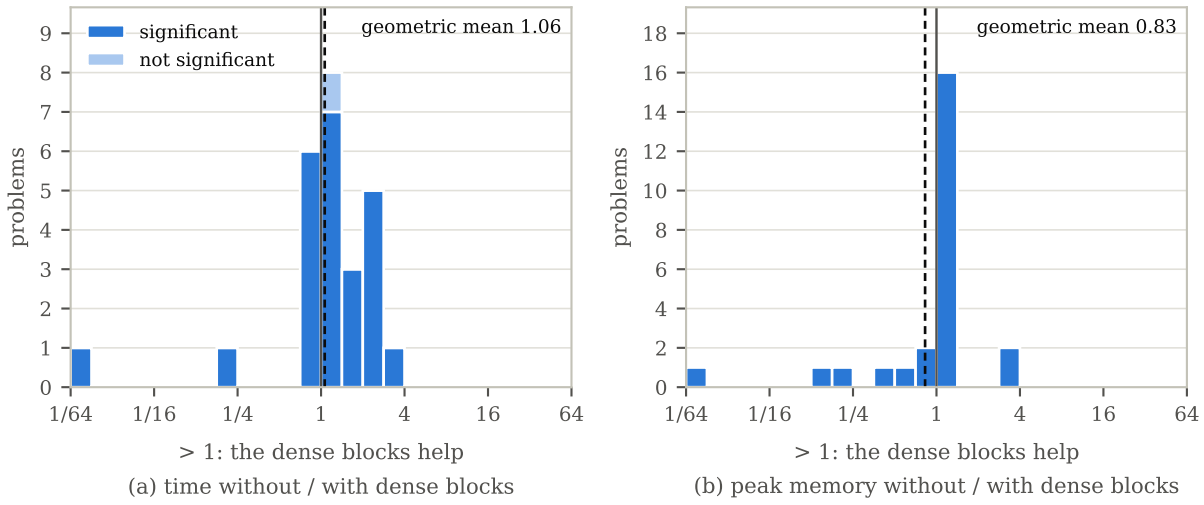


Figure 2: Figure 4.3 of the thesis (p. 58): distribution over the 25 problems of the ratio without / with the dense blocks, in time (a) and peak memory (b). The dashed line is the geometric mean.

Table 2: The ablations in summary: geometric mean of the ratio of each variant to its reference, its bootstrap 95 % confidence interval, the Wilcoxon p-value over the problems, and the number of problems with a significant difference (Mann–Whitney, Holm-adjusted).

Variant	Reference	Ratio	95 % CI	p	Significant
First canonicalization, 25 problems					
no dense blocks	`diffengine`	1.06	0.71–1.45	0.067	24/25
no dense blocks	best `CVXPY`	0.46	0.36–0.61	3.2×10^{-5}	25/25
Re-solve, 7 DPP problems					
no dense blocks	`diffengine`	1.01	0.98–1.05	0.938	6/7
no split	`diffengine`	2.86	1.73–4.91	0.016	7/7
neither	`diffengine`	3.68	2.19–6.20	0.016	7/7

Q3. *How does re-solving behave on a non-DPP parametric problem?*

On the two families written with a product of two parameters, which DPP forbids, the cached re-solve of `diffengine` is 13 to 14× faster than `CVXPY` on `lasso` and `factor_cov`, a significant difference on both (Table 3), because `CVXPY` has to canonicalize the problem again at every solve. ConvexPlasticity, the suite's only natural non-DPP parametric problem, is the exception: it re-solves in 207 ms against 42 ms for `CVXPY`. The split is not the cause: 88 % of that re-solve assembles the dense 3000×3000 block of $\mathbf{P}$, built from a diagonal given as a dense array. Without the dense blocks, the same cached re-solve takes 2.0 ms, 21× faster than `CVXPY`. The split therefore carries over to a real non-DPP problem; what fails there is the dense storage of a constant that is sparse in fact.

The non-DPP experiment is new in the thesis. The thesis also no longer claims that the split replaces the tensor: the discussion now says that the split re-solves within a small factor of the tensor without requiring DPP, and much faster than `CVXPY` when the problem is not DPP.

Table 3: Table 4.7 of the thesis (p. 67): re-solve time in milliseconds, mean over 30 runs. `CVXPY` applies the tensor on the DPP variant and canonicalizes again on the others; the cached column of `diffengine` runs the numeric pass only. The ratio is the median of `diffengine`, cached when it can, over that of `CVXPY`.

			`diffengine`		
Family	Variant	`CVXPY`	cached	rebuild	Ratio
`lasso`, $n = 1000$	DPP	6.4	23.3	32.7	3.64*
	non-DPP	315	23.3	32.8	0.07*
	parameter-free	338	—	49.0	0.14*
`factor_cov`, $n = 5000$	DPP	1.2	3.6	6.9	2.86*
	non-DPP	45.8	3.6	7.0	0.08*
	parameter-free	47.1	—	11.0	0.23*

Changes in the thesis, §4.5.1, p. 66

On the non-DPP variants, `CVXPY` canonicalizes the problem again at every solve, while `diffengine` runs the same numeric pass as on the DPP variant and is 13 to 14 times faster. On the DPP variants, the tensor is 3.6 and 2.9 times faster. ConvexPlasticity, the only parametric problem of the suite that is not DPP, is the exception: `diffengine` re-solves it in 207 milliseconds against 42 for `CVXPY`, because 88 % of the re-solve assembles the dense 3000×3000 block of $\mathbf{P}$. Without the dense blocks, it re-solves in 2.0 milliseconds, 21 times faster than `CVXPY`.

Changes in the thesis, §4.6, p. 67

Caching canonicalization does not necessarily require the DPP ruleset. [...] the symbolic and numeric split re-solves within a factor of four of the cached tensor on every DPP problem. [...] it re-solves problems that are not DPP, on which `CVXPY` has to canonicalize again, 13 to 14 times faster.

Q4. *Why not re-evaluate only the subtrees that depend on the updated parameters?*

The engine now does this: after a parameter update, the numeric pass re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others. All re-solve times in the thesis use it.

On the seven DPP problems (Table 4), `diffengine` has the fastest first canonicalization on all seven, 2.5 to 6.6× faster than the fastest tensor build. The cached tensor re-solves faster on four, by 1.10 to 3.60×, and `diffengine` on the other three. On the four, the numeric pass takes only 4 to 24 % of a re-solve; the rest converts the output to compressed sparse columns in the cone order of the solver, which the tensor produces directly. The tensor recovers its slower first canonicalization only after 18 to 124 re-solves. On problems that are not DPP (Q3), `CVXPY` has no tensor and `diffengine` is 13 to 14× faster. *We conclude that the split does not replace the tensor on DPP problems that are re-solved many times, but that it comes within a factor of four of it without the DPP rules, and that its remaining cost is the conversion to the solver's format, not the numeric pass.* Making this conversion cheaper is now the first item of future work.

Table 4: Table 4.5 of the thesis (p. 65): first canonicalization in seconds and re-solve in milliseconds, mean over 30 runs. A dash is a tensor that exceeded the memory or time limit. The ratio is the median re-solve of `diffengine` over that of the fastest tensor, * when significant.

	first canonicalization (s)				re-solve (ms)		
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	2.02	7.30	0.083	**0.033**	1.8	**0.4**	0.24*
ParamConeMatrixStuffing	0.74	1.85	0.055	**0.022**	1.0	**0.3**	0.27*
SimpleFullyParametrizedLP	—	—	0.57	**0.12**	47	**35**	0.73*
SimpleScalarParametrizedLP	1.33	1.17	1.19	**0.23**	**63**	70	1.10*
ParametrizedQP	—	—	2.63	**0.76**	**299**	349	1.16*
SVMWithL1Regularization	2.80	2.57	2.83	**0.39**	**38**	125	3.26*
FactorCovarianceModel	1.20	1.14	1.19	**0.18**	**21**	74	3.60*

Changes in the thesis, §3.4.1, p. 47

The numeric pass therefore skips an affine tree after the first evaluation, until the next parameter update. After an update, it re-evaluates only the subtrees that contain a parameter, and keeps the cached values of the others.

Q5. *When does the global layout become a handicap relative to `CVXPY`'s local layout, in particular for memory?*

When the problem has many variables and many nodes. In the global layout, each node keeps, besides its Jacobian, integer work arrays of length n, the total number of variables, whatever variables its own subtree contains. Their memory is therefore $O(\text{nodes} \times n)$ integers. In `CVXPY`'s local layout, each node stores blocks only for the variables it depends on, so the same overhead is proportional to the sum, over the nodes, of the number of variables each

one depends on. The two coincide when every node depends on every variable, and the local layout wins when most nodes depend on a few variables out of many.

OptimalAdvertising shows it. Each of its 250 minimum terms depends on one row of a 250×1000 variable, but has a Jacobian over all 250 000 variables, and the symbolic pass visits every column. Without the dense blocks, its peak memory is 945 MB, against 173 to 197 MB for the `CVXPY` backends; that difference is the per-node cost. With the dense blocks it reaches 6.1 GB, and most of the difference comes from the aggregate constraints $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ and $\mathbf{D1} \geq \mathbf{c}$. Canonicalized alone, $\mathbf{D}^\top \mathbf{1} \leq \mathbf{t}$ takes 4 GB with the dense blocks and 112 MB without. Its Jacobian is stored as 1000 dense blocks of 250 entries, but each block also keeps an inverse map of its columns with one integer per variable of the problem, 250 000 of them. This is the cost of the global layout again, paid once per dense block rather than once per node. Storing these maps sparsely, or sharing one workspace across the blocks, should remove most of it.

Both aggregate constraints are sums written as products with a vector of ones. The converter from `CVXPY` to the engine can recognize a product $\mathbf{D1}$ or $\mathbf{D}^\top \mathbf{1}$ and build the engine's sum atom instead, which builds no dense block. With this rewrite, OptimalAdvertising canonicalizes in 0.33 s instead of 0.91 s, faster than the 0.53 s of the fastest `CVXPY` backend, with a peak memory of 1.4 GB instead of 6.1 GB (medians of 30 runs each). A weighted sum $\mathbf{Dw}$ with a general dense $\mathbf{w}$ still needs the fix in the engine.

Changes in the thesis, §5.1, p. 69

Another part of the memory cost comes from the global layout. Every node of the expression trees holds a Jacobian over all n variables, together with integer work arrays of length n. The values stored remain proportional to the nonzeros, but this overhead grows with the number of nodes times the number of variables. On problems with very many variables and deep trees, it can exceed the memory of `CVXPY`'s local layout, whose blocks span only the variables that each node depends on.

Q6. *What part of* ***`SparseDiffEngine`*** *pre-existed the thesis or was developed for DNLP, and what part is specific to this work?*

Daniel Cederberg leads `SparseDiffEngine`. He started it as the differentiation engine of the `DNLP` extension of `CVXPY` [5], which differentiates nonlinear atoms at every iterate of the solver, and I co-developed it with him. The version sent to the committee should have said so clearly; it now does. This thesis uses the engine for a different purpose, canonicalization: it evaluates the affine atoms and the quadratic form at the origin to obtain the problem data. What is specific to the thesis is the interface to `CVXPY`, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Changes in the thesis, §1.5, p. 16

Daniel Cederberg is the lead developer of `SparseDiffEngine`. It was first built as the differentiation engine of the `DNLP` extension of `CVXPY`, where it differentiates nonlinear atoms such as `exp`, `log` and `entr` at every iterate of the solver. This thesis builds on the observation that canonicalization is a special case of the same computation: once a problem is canonicalized,

only affine atoms and quadratic forms remain, and their derivatives are the problem data.

Changes in the thesis, §3.1.1, p. 37

What is specific to this thesis is the interface to `CVXPY` of §3.3, namely the `diffengine` backend and its re-solve path for parametric problems, and the benchmarks of Chapter 4.

Q7. *What is the strategy for BLAS and binary distribution of* ***`sparsediffpy`****?*

`cvxcore` is a C++ extension, built with a copy of `Eigen`, that lives inside the `CVXPY` repository, so `CVXPY` has to compile and ship it. `sparsediffpy` is a separate package: `CVXPY` only declares it as a dependency and calls it, which keeps the `CVXPY` side of the backend much simpler. `sparsediffpy` is still compiled, in C, and its own wheels are built for Linux, macOS and Windows and published on `PyPI`. They link Apple's `Accelerate` on macOS and bundle `OpenBLAS` on Linux and Windows, so users install nothing beyond the package. The thesis did not say this; it now does, in the section that introduces `sparsediffpy`.

Changes in the thesis, §3.4, p. 47

Unlike `cvxcore`, which is compiled inside the `CVXPY` repository, `sparsediffpy` is a separate package on which `CVXPY` only depends. Its compiled wheels for Linux, macOS and Windows link Apple's `Accelerate` on macOS and a bundled copy of `OpenBLAS` on the other two, so users need no BLAS of their own.

3 Comments on the thesis

This section answers the comments written on the October 24 diff of the thesis, in the order of the pages. Comments already answered by the restructuring of Chapter 4 (moving the appendix results into the chapter, the figures for size and density, and 30 runs everywhere) are grouped at the end.

Declaration on the use of AI. The acronym is now defined at its first use, "since" became "because", and the experiments paragraph now says what was verified ("tested what they were meant to test", with two examples). The appeal to software engineering is replaced by the practice of the two projects themselves. To your three questions on page ix: I read the `.tex` source of the text and made the final small edits myself; your corrections are applied one by one, by hand or through Claude Code, and I check each of them in the rebuilt PDF and in the diff; and I proofread the whole text before sending it, so you are not its first reader. The declaration now says so.

Changes in the thesis, Declaration on the use of AI, p. ix

I verified that the resulting PDF matches what I expected or wanted, and looked less at the `.tex` source of the figures. For the text, I read the `.tex` source and made the final small edits myself. The corrections of my advisor were applied one by one, either by hand or through

Claude Code, and I checked each of them in the rebuilt PDF and in the difference with the previous version.

Chapters 1 to 3.

- The parenthesis of §1.1 is now a sentence: "Note that in doing so, n also grows."
- "But you are the lead?" (§1.5): Daniel Cederberg leads `SparseDiffEngine`; Q6 above explains the split, and §1.5 now states it explicitly.
- The assembly of finite-element matrices (§3.2) now cites Hughes's textbook. The renaming of `CVXcanon` to `cvxcore` in 2018 is a fact of the project's history that needs no citation.
- "Diamond and Boyd" now carries the citation directly (§2.2), the tangents, adjoints and accumulations of Figures 2.7 to 2.9 use more saturated colours, and the serial comma and the spacing before `.nl` are fixed. The period after "MOI [90]" was an artefact of the diff and is in place in the thesis.

Chapter 4.

- The merge of the backend into `CVXPY` and my contributions to the `SCIPY` and `COO` backends moved from the setup to the future work of the conclusion, where they support the adoption of `diffengine`.
- *Why Clarabel?* The target solver only fixes the form of the output; Clarabel is the default conic solver of `CVXPY` and accepts every cone of the suite, so every problem is canonicalized to the same form.
- *What do we get from the tests?* The statistics paragraph and the caption of Table 4.1 now say what a significant difference means and why the Holm adjustment is needed.
- *30 re-solves?* Every measurement is made in 30 runs; each run of a parametric problem performs five re-solves and keeps their median, so every re-solve time is also a distribution over 30 runs. The text now says this, and that the re-solve times cover the whole call to `get_problem_data`, including the conversion to the solver's format.

Changes in the thesis, §4, Statistics

A significant difference on a problem means that the gap between the two backends is too large to be explained by the variation from one run to the next. Because we test 25 problems at once, some of them would appear significant by chance alone if each were tested at 5 %; the Holm–Bonferroni adjustment keeps the probability of even one such false result below 5 % over the whole comparison, so each significant ratio can be read on its own.

Changes in the thesis, §4, Setup

The target solver only fixes the form of the output. We choose `Clarabel` because it is the default conic solver of `CVXPY` when no commercial solver is installed, and because it accepts a quadratic objective and every cone of the suite, so every problem is canonicalized to the same form (1.2) without reductions specific to a solver.

Already addressed. The run-in heading "The four slower problems" is now §4.4, p. 62; the ablation, the size and density results (with Figures 1–4 as visual representations) and their tables are in Chapter 4 instead of Appendix A; and every measurement is made in 30 runs, apart from three cells of the size sweep that take more than 30 seconds per run. The diff now marks additions in plain blue, without the wavy underline.

4 Further results

4.1 First canonicalization

`diffengine` is the fastest of the four backends on 21 of the 25 problems, and the difference is significant on all 21 (Table 5). The geometric mean of its ratio to the best `CVXPY` backend is 0.44, with a bootstrap confidence interval of 0.29 to 0.70, and it differs significantly from 1 (Wilcoxon, $p = 0.003$). On 15 problems it is more than twice as fast as the best `CVXPY` backend, and it is significantly slower on four (Section 4.3). The separation is complete: on every problem, every run of `diffengine` is faster than every run of the best `CVXPY` backend, or slower than every one, so the confidence intervals of Figure 1 (in Q2) are narrower than the markers. *We conclude that the speedup on the first canonicalization is real and not an artefact of a few problems:* 21 *of* 25 *problems, none of them close to a tie.* In peak memory, `diffengine` is the lowest of the four on 12 problems, and in geometric mean it uses 1.04 times the memory of `CPP` and 1.45 times that of `SCIPY`, so it is on par with the default backend but not with the best one.

4.2 Sensitivity to size and density

Size. All 25 problems and the parameter-free `lasso` family are measured at $s \in \{1/4, 1/2, 1, 2, 4\}$, each cell in 30 runs with its own Mann–Whitney test, Holm-adjusted within each size. Figure 3 replaces the two size tables of the earlier version, and every one of its 119 measured cells is significant. On 18 of the 22 problems where `diffengine` is faster at $s = 1$, the ratio stays within 20 % of its value at $s = 1$ over a factor of 16 in size. Of the four slower problems, UnconstrainedQP (5.7 to 22.6), ConvexPlasticity (3.3 to 8.8) and OptimalAdvertising (0.80 to 3.0) get worse with the size, for the reasons of Section 4.3, and SDPSegfault1132 improves slightly. `diffengine` alone exceeds the 20 GB memory limit in five cells, all at $s \geq 2$, and every backend exceeds it in four more. *We conclude that the speedup does not depend on the size chosen by the benchmark suite, and that the slowdowns and the memory cost of the*

Table 5: Table 4.1 of the thesis (p. 57): first canonicalization in milliseconds, mean ± standard deviation over 30 runs. Bold is the fastest backend; the ratio is the median of `diffengine` over that of the best `CVXPY` backend, * when significant (Mann–Whitney, Holm-adjusted $p < 0.05$).

Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	Ratio
SVMWithL1Regularization	2 786 ± 8	2 634 ± 8	2 742 ± 15	**387 ± 1**	0.15*
FactorCovarianceModel	1 223 ± 5	1 164 ± 3	1 150 ± 4	**177 ± 3**	0.15*
Yitzhaki	938 ± 1	982 ± 2	1 006 ± 3	**145 ± 1**	0.15*
SimpleQP	2 891 ± 5	2 944 ± 6	2 929 ± 6	**453 ± 1**	0.16*
HuberRegression	2 770 ± 6	2 850 ± 7	2 796 ± 7	**495 ± 8**	0.18*
LeastSquares	2 410 ± 9	2 912 ± 7	2 939 ± 5	**435 ± 2**	0.18*
SimpleLP	6 898 ± 38	6 303 ± 17	5 962 ± 14	**1 121 ± 3**	0.19*
SimpleScalarParametrizedLP	1 260 ± 3	1 151 ± 2	1 087 ± 3	**230 ± 1**	0.21*
SimpleFullyParametrizedLP	611 ± 1	555 ± 1	523 ± 1	**115 ± 0.5**	0.22*
SlowPruning	1 792 ± 4	2 185 ± 4	2 147 ± 4	**408 ± 1**	0.23*
ParametrizedQP	2 974 ± 29	3 030 ± 7	2 990 ± 10	**755 ± 46**	0.25*
Cajas	3 885 ± 12	689 ± 5	693 ± 5	**205 ± 1**	0.30*
ConeMatrixStuffing	2 517 ± 10	2 152 ± 8	3 537 ± 8	**642 ± 4**	0.30*
SmallMatrixStuffing	1 683 ± 13	1 822 ± 8	2 508 ± 7	**502 ± 2**	0.30*
TvInpainting	919 ± 1	913 ± 2	815 ± 2	**257 ± 1**	0.32*
ParamSmallMatrixStuffing	63.3 ± 0.2	127 ± 1	113 ± 1	**32.7 ± 0.1**	0.52*
ParamConeMatrixStuffing	42.8 ± 0.2	71.9 ± 0.3	88.7 ± 0.4	**22.4 ± 0.4**	0.52*
CVaR	7 196 ± 15	6 503 ± 20	6 512 ± 23	**3 421 ± 9**	0.53*
SemidefiniteProgramming	1 161 ± 3	632 ± 2	997 ± 4	**405 ± 3**	0.64*
QuantumHilbertMatrix	1 062 ± 2	805 ± 4	788 ± 2	**653 ± 3**	0.83*
Murray	1 276 ± 18	1 229 ± 7	1 224 ± 7	**1 181 ± 1**	0.96*
SDPSegfault1132	8 000 ± 11	**1 663 ± 148**	7 298 ± 110	2 147 ± 6	1.32*
OptimalAdvertising	2 573 ± 3	598 ± 8	**534 ± 1**	901 ± 4	1.68*
ConvexPlasticity	78.2 ± 0.7	72.0 ± 0.7	**69.8 ± 1.2**	507 ± 1	7.31*
UnconstrainedQP	**1 955 ± 3**	1 980 ± 5	2 439 ± 2	22 937 ± 21	11.73*
Geometric mean					**0.44**

global layout grow with it.

Density. The data matrix of eight problems is masked to a density $\rho \in \{0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1\}$ and kept as a dense array, as a user would pass it. Without the dense blocks, `diffengine`'s advantage shrinks slowly: on six of the eight problems, the ratio to the best `CVXPY` backend rises from 0.28–0.38 on dense data to 0.52–0.79 at $\rho = 0.01$. With them, it starts lower, 0.14–0.18, but rises much faster, because a dense block stores the zeros as well. The two curves cross between $\rho = 0.5$ and $\rho = 0.2$, and at $\rho = 0.05$, just above the 5 % threshold below which the engine switches to the sparse format, `diffengine` is slower than `CVXPY` on five of the eight problems. Below the threshold the two curves meet. Every point of Figure 4 is significant. *We conclude that the dense blocks pay off on data that is dense in fact, and slow* ***`diffengine`*** *down on data that is mostly zeros but stored as a dense array. Without them,* ***`diffengine`*** *stays faster than* ***`CVXPY`*** *down to* 1 % *on six of the eight problems.*

Changes in the thesis, §4.3, p. 60

On HuberRegression and SemidefiniteProgramming, `diffengine` is slower at the lowest densities whether or not it builds the dense blocks, and on SemidefiniteProgramming the dense blocks make no difference at any density. The dense blocks therefore pay off only on data that is dense in fact.

4.3 The four slower problems

Table 6 gives the share of each phase on the four problems where `diffengine` is slower. Each has one cause, and three of the four have a fix outside the engine.

- **UnconstrainedQP** (11.7× slower): 98 % of the time is the numeric pass, filling a dense 63 504 × 36 Jacobian of $(\mathbf{F}^{\mathsf{H}}\mathbf{E})\mathbf{F}$ evaluated from the left. Evaluated from the vector $\mathbf{x}$, every intermediate is a vector and the canonicalization takes 16 ms instead of 22.9 s. This reassociation speeds up every backend, so none of them uses it in the thesis.
- **ConvexPlasticity** (7.3×): the diagonal $h\mathbf{I}$ is given as a dense array and becomes a dense 3000 × 3000 block of $\mathbf{P}$ (47 % symbolic, 38 % assembly). Storing $\mathbf{P}$ sparse below 5 % nonzeros, as is already done for the factors of a product, reduces the canonicalization from 509 to 203 ms, still 2.9× the fastest `CVXPY` backend; most of the remainder is the symbolic pass.
- **OptimalAdvertising** (1.7×): 89 % is the symbolic pass, which visits all 250 000 variables for each of the 250 minimum terms; this is the global layout of Q5. Rewriting the products with a vector of ones as sums reduces it from 0.91 to 0.33 s, faster than `CVXPY`.
- **SDPSegfault1132** (1.3×): three dense blocks of 49.5 million entries cancel in a sum with 1.01 million nonzeros, and both passes carry the cancelled entries. This one has no fix yet.

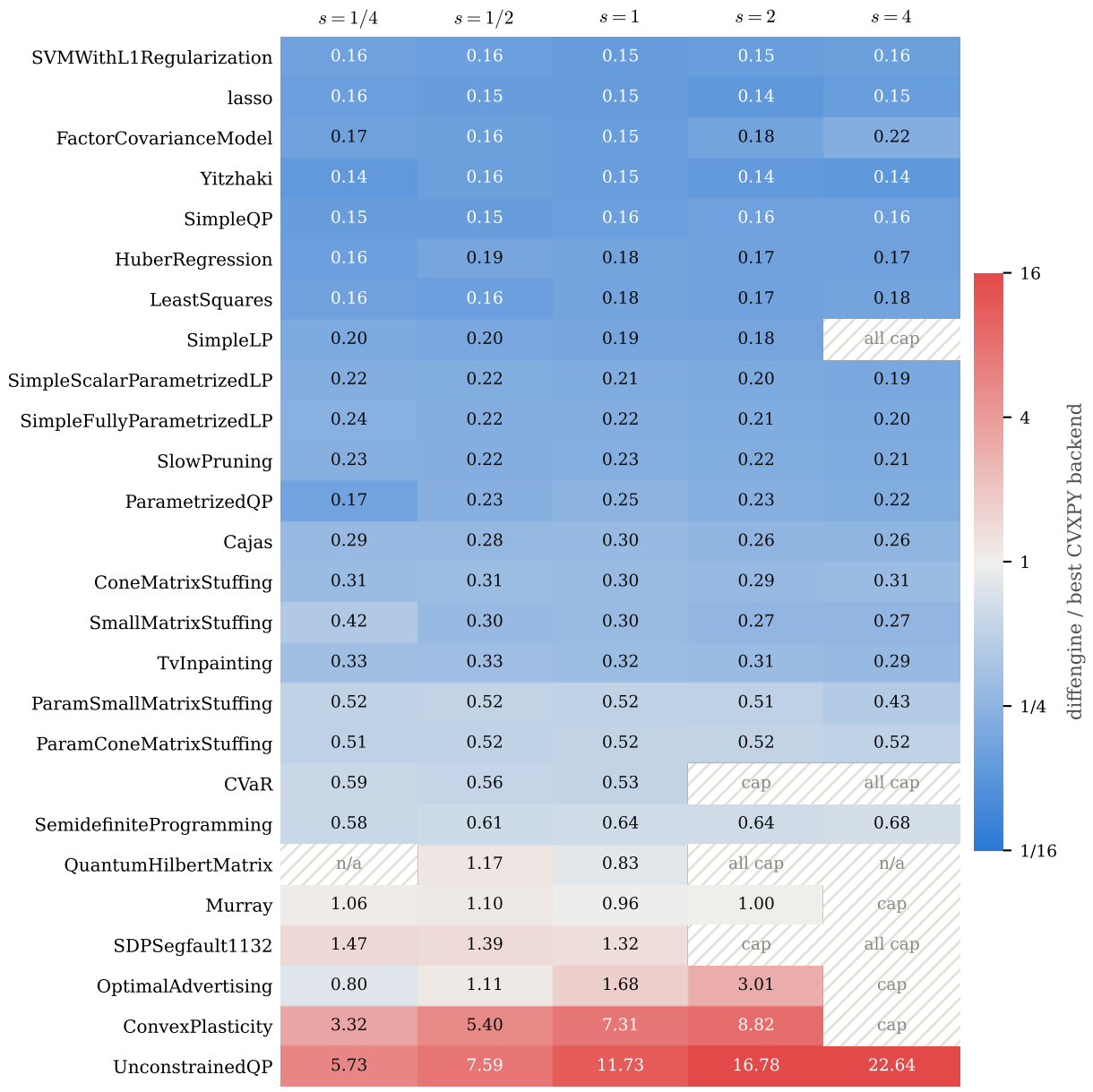


Figure 3: Figure 4.4 of the thesis (p. 61): ratio of `diffengine` to the best `CVXPY` backend as each problem is scaled by s. Every measured cell is significant. "cap": `diffengine` exceeded 20 GB while a `CVXPY` backend finished; "all cap": every backend exceeded it.

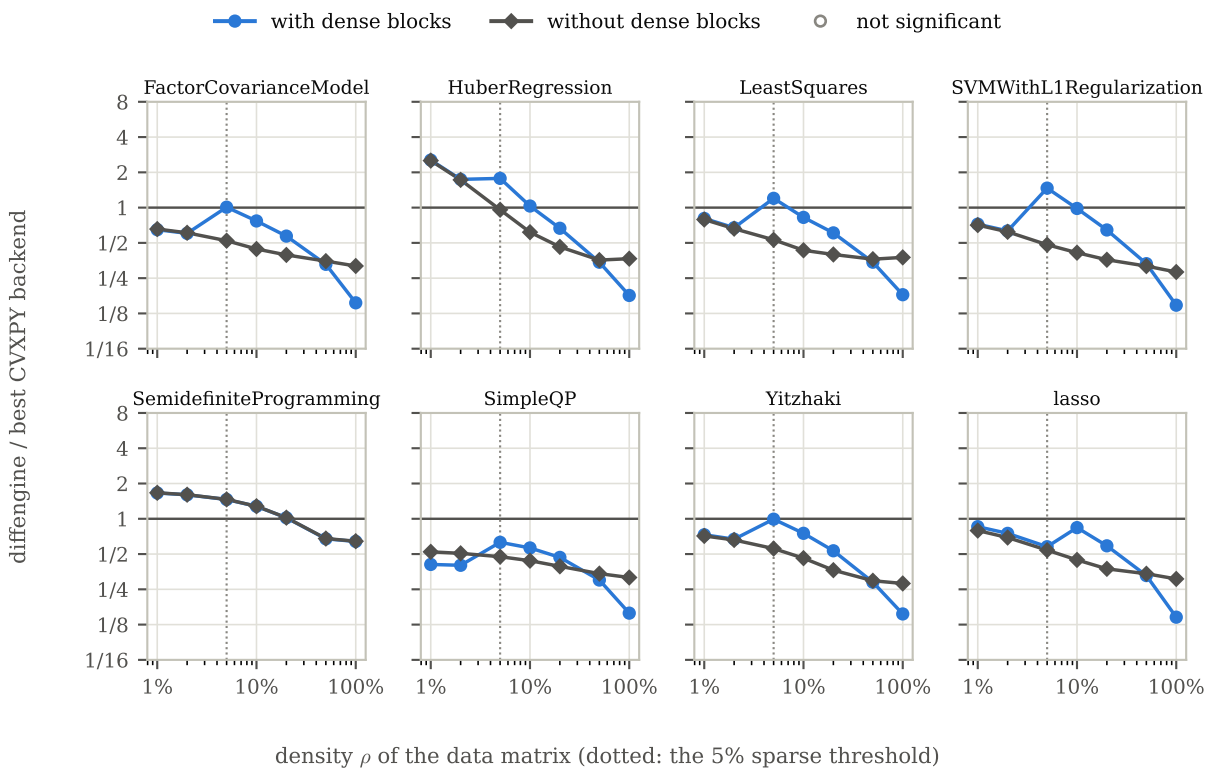


Figure 4: Figure 4.5 of the thesis (p. 62): ratio of `diffengine` to the best `CVXPY` backend as the density of the data falls, with and without the dense blocks. The dotted line is the 5 % sparse threshold.

Each fixed time is the median of 30 runs with the fix applied alone to the version of `CVXPY` used in the thesis, measured in the same session as 30 new runs without it. *We conclude that the slowdowns come from how the data reaches the engine, not from the split or the C implementation. On two of the four, UnconstrainedQP and ConvexPlasticity, the cause is a dense block built from sparse data, and without the dense blocks both are faster than on any* `CVXPY` *backend or within* 1.4× *of it (Figure 1).*

Table 6: Table 4.4 of the thesis (p. 63): phases of the first canonicalization of `diffengine`, in percent of its time.

Benchmark	Time (s)	Front end	Convert	Symbolic	Numeric	Assemble	Format
UnconstrainedQP	22.94	0	0	2	98	0	0
ConvexPlasticity	0.51	6	2	47	6	38	0
OptimalAdvertising	0.90	4	1	89	4	1	1
SDPSegfault1132	2.15	0	0	49	47	3	0
Median, all 25	—	4	9	19	14	12	12

5 Discussion

The results support three observations, which the discussion of the thesis (§4.6, p. 67) now states with these numbers.

1. *Caching does not require DPP.* The split re-solves within a factor of four of the tensor on every DPP problem, and 13 to 14× faster than `CVXPY` on problems that are not DPP.

2. *The tensor has a large upfront cost.* Its first canonicalization is slower on all seven DPP problems, and on the four where it re-solves faster, it pays off only after 18 to 124 re-solves.

3. *`cvxcore` can be replaced.* `diffengine` is faster than `cvxcore` on 23 of the 25 problems, and `COO` builds the tensor faster than `cvxcore` on six of the seven.

The claims have also become narrower than in the version sent to the committee. The thesis no longer says that the split replaces the tensor, and it states the four slower problems, the memory cost of the global layout and the conversion cost of the re-solves as limitations, with the fixes that remain to be made.

References

[1] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

[2] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[3] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[4] S. Diamond and S. Boyd, "Matrix-free convex optimization modeling," in *Optimization and Its Applications in Control and Data Sciences*, ser. Springer Optimization and Its Applications, B. Goldengorin, Ed. Springer, 2016, vol. 115, pp. 221–264.

[5] D. Cederberg, W. Zhang, P. Nobel, and S. Boyd, "Disciplined nonlinear programming," arXiv:2606.02896, 2026. [Online]. Available: https://arxiv.org/abs/2606.02896